

# DirectX12を使用した自作エンジンで作った3Dアクションゲーム

|       |                         |
|-------|-------------------------|
| 制作人数  | 1人                      |
| 使用ツール | VisualStudio2022、GitHub |
| 使用言語  | C/C++                   |
| 制作期間  | 2025/7~現在まで             |

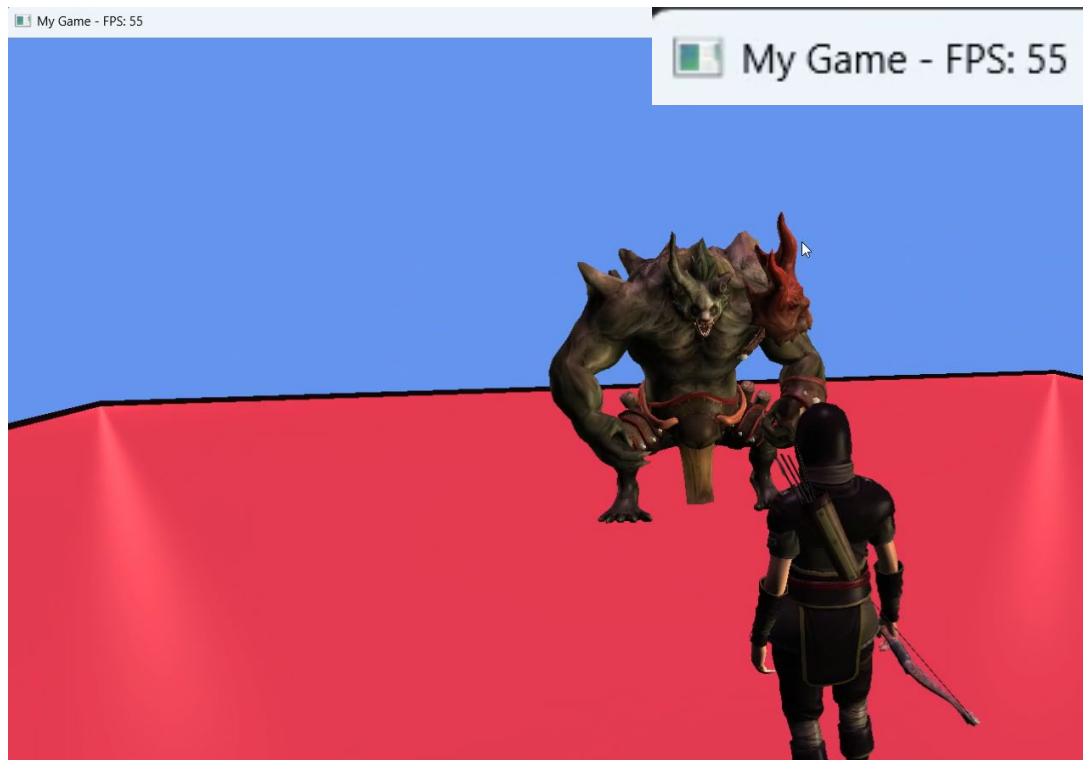
## 制作目的

学校の授業で製作しているオブジェクト指向のDirectX12のエンジンがあったので、それを参考にしながら別のやり方でやってみたいと思い、ECSを使用して作り変えてみるのが目的で始めました。

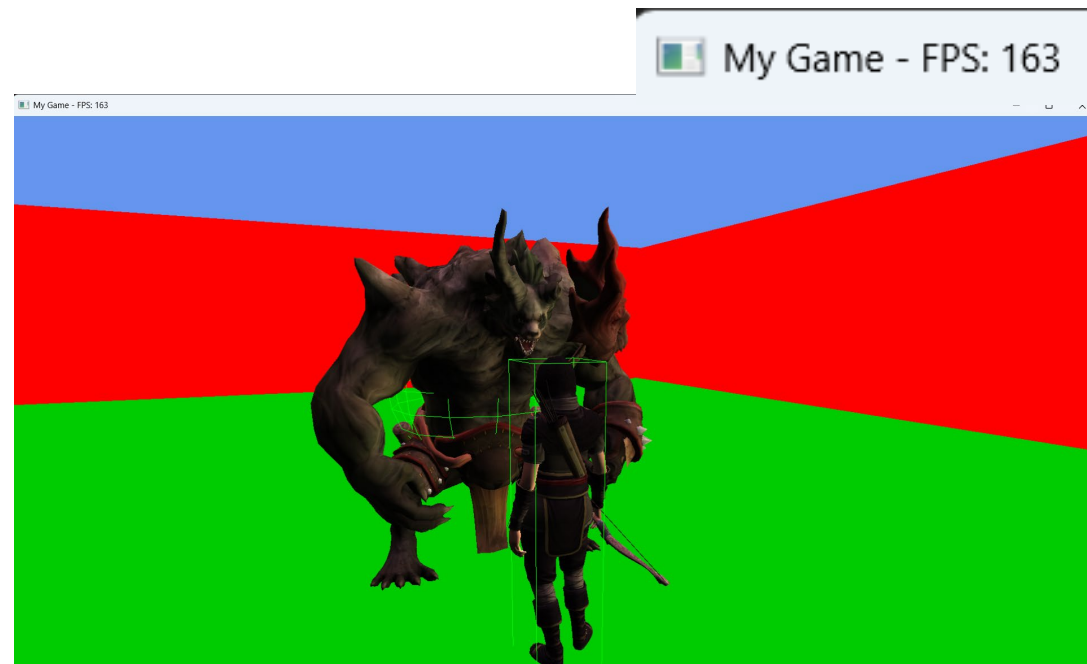
# パフォーマンスを向上させました。

最初はシンプルなECSを実装していたが、コンポーネントを個別に管理していたため、同一Entityのデータがメモリ上に連続しておらず、キャッシュミスが頻発して処理が重くなっていました。これを解決するために、Archetypeを導入したことで、同一のコンポーネントセット(Archetype)ごとに専用の連続メモリを確保し、高速化に成功しました。

## アーキテクチャ実装前



## アーキテクチャ実装後



# 三角形と光線の当たり判定を実装した。

画像のように三角形と光線の当たり判定を利用して、壁などの障害物がプレイヤーとの間にあるときに、光線が当たった場所にカメラを移動させることでカメラが障害物を貫通しないようにした。

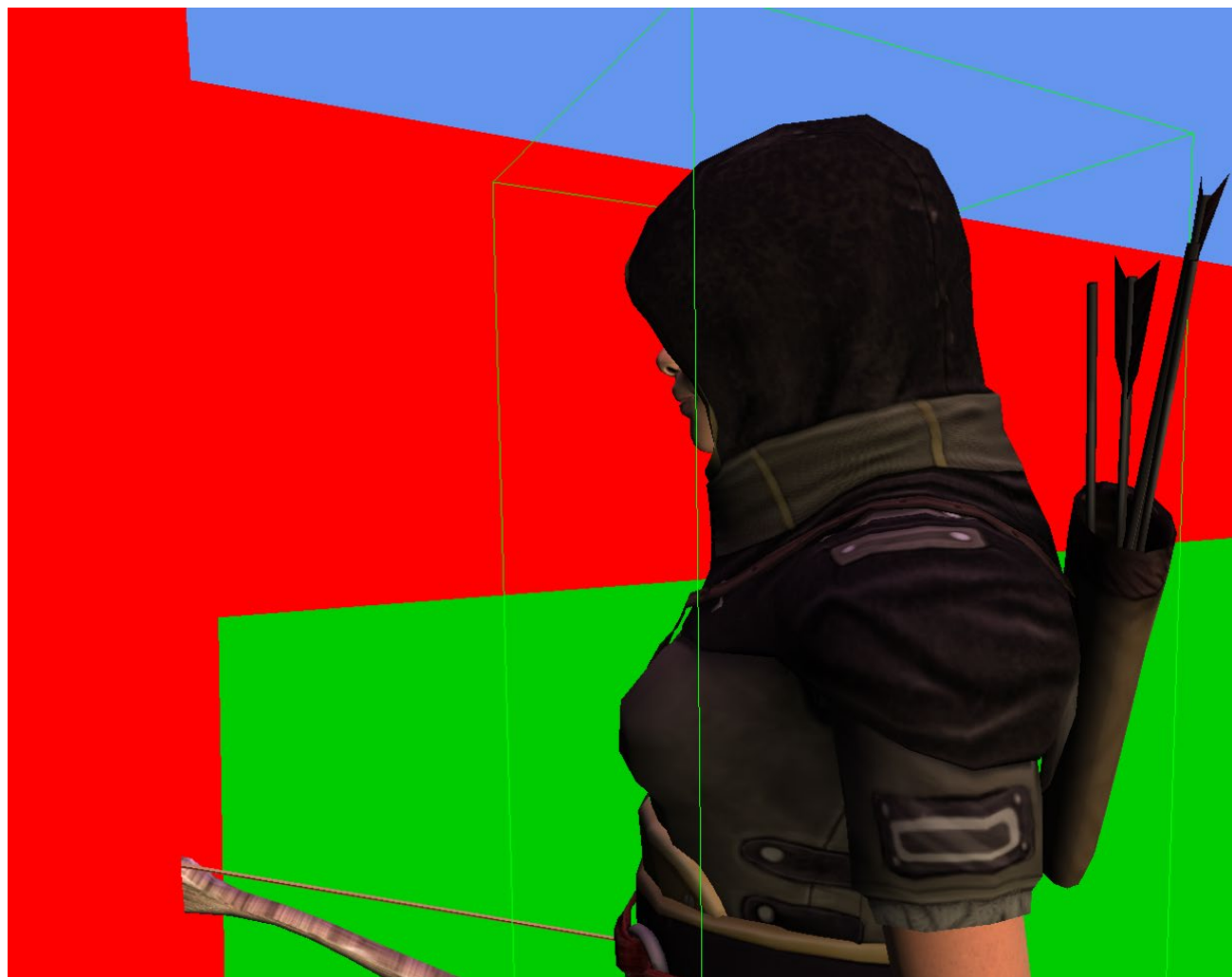
また、この時に障害物を判定させるためのビットフラグを使用したレイヤーマスク機能も制作した。

```
// レイヤー定数の定義(32個まで)
namespace Layers
{
    // 既定のレイヤー
    constexpr LayerMask Default = (1 << 0);
    constexpr LayerMask TransparentFX = (1 << 1);
    constexpr LayerMask IgnoreRaycast = (1 << 2);

    constexpr LayerMask Water = (1 << 4);
    constexpr LayerMask UI = (1 << 5);

    // ユーザー用定義レイヤー
    constexpr LayerMask Environment = 1 << 8;

    // 全レイヤー
    constexpr LayerMask Everything = 0xFFFFFFFF;
}
```



# 使いやすいダメージシステム①

敵の情報を知らなくてもダメージを与えたり、  
一回の攻撃で同じ敵に何度もダメージを与えてしまうのを防いだりする  
汎用的なシステムを作成しました。

# 使いやすいダメージシステム②

## Attackable編

何かにダメージを与えたいEntityにAttackableコンポーネントを付けます。

Attackableコンポーネントでは、現在攻撃中かを確認するisAttackingと攻撃したEntityを格納するentitiesを持ちます。  
(画像②)

それぞれの攻撃システムでは、攻撃中にisAttackingをtrueにし、entitiesに攻撃をしたEntityを格納することで、ダメージを与える際に、既に攻撃済みのEntityを知ることができ、一回の攻撃で同じものに複数回ダメージを与えてしまうことがなくなります。  
(画像③)

```
struct Attackable : IComponentData
{
    bool isAttacking = false;
    std::vector<Entity> entities;
};
```

(画像①)

[DirectX12Engine/DirectX12Engine/DirectX12Engine/Attackable.h at main · sumidayuki/DirectX12Engine](#)

```
void AttackableSystem::Update(World& world)
{
    View<Attackable> view(world)

    for (auto [entity, attackable] : view)
    {
        if (attackable.entities.empty())
        {
            continue;
        }

        if (!attackable.isAttacking)
        {
            attackable.entities.clear();
        }
    }
}
```

(画像②)

[DirectX12Engine/DirectX12Engine/DirectX12Engine/AttackableSystem.cpp at main · sumidayuki/DirectX12Engine](#)

# 使いやすいダメージシステム③

## Damageable編

ダメージを受けたいEntityにこのコンポーネントをアタッチします。

DamageableコンポーネントはDamage構造体を格納することができるdamageQueueだけを所持しています。(画像①)

ダメージを与える側のプレイヤーがダメージを与えたいEntityのDamageableコンポーネントを取得し、Damageを作成してDamageableにpushすることで、ダメージを受ける側はこの中身を一つずつpopするだけ良い仕様になっています。(画像②)

```
enum DamageType
{
    Normal,
};

struct Damage
{
    DamageType type;
    int damage;
};

struct Damageable : IComponentData
{
    std::queue<Damage> damageQueue;
};
```

(画像①)

[DirectX12Engine/DirectX12Engine/DirectX12Engine/Damageable.h at main · sumidayuki/DirectX12Engine](#)

```
if ((coll.info.state == CollisionState::Enter || coll.info.state == CollisionState::Stay))
{
    if (world.Component<Enemy>(coll.info.other))
    {
        attackable.isAttacking = true;
        for (int i = 0; i < attackable.entities.size(); i++)
        {
            if (attackable.entities[i] == coll.info.other)
            {
                return;
            }
        }

        Damageable* damageable = world.GetComponent<Damageable>(coll.info.other);
        if (damageable)
        {
            Damage damage;
            damage.type = DamageType::Normal;
            damage.damage = 10;
            damageable->damageQueue.push(damage);
            attackable.entities.push_back(coll.info.other);
            world.DestroyEntity(entity);
        }
    }
}
```

(画像②)

[DirectX12Engine/DirectX12Engine/DirectX12Engine/ArrowSystem.cpp at main · sumidayuki/DirectX12Engine](#)

# 一番工夫した敵AIシステム①

最初は、ステートのような形で一つ一つの行動を制作していましたが、データ駆動型のBehaviourTreeやコンボシステムなどを組み合わせたことにより、状況に応じた柔軟な動きを簡単に追加することができるようになり敵のAIに幅が広がりました。

# 一番工夫した敵AIシステム②

敵AIを作る上で、一つ一つの行動や考えに意味を持たせることを意識して制作しました。わかりやすい行動として、攻撃後に休憩をはさむのですが、この際に、プレイヤーの方向を常に向くのではなく、プレイヤーが視界外に出そうになったら、プレイヤーの方向に向くようにしました。これにより、生き物感のある動きになり自然なゲーム体験ができるようになりました。

視界内



視界外

